SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192372087
Name	Atmakuri Sai Mrudula

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**

Question Count: 5

Code of Conduct

I **Atmakuri Sai Mrudula (192372087)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **A. Sai Mrudula**

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

A Hadoop job repeatedly fails because the input splits are uneven, causing some reducers to receive a large amount of data while others remain idle. This results in poor load balancing and inefficient execution of the Hadoop job.

Error Analysis

The main design error is improper data partitioning, which leads to data skew.

- Hadoop divides large input files into smaller input splits for parallel processing.
- If the splits are not balanced, some tasks receive significantly more data than others.
- During reduce phase, some reducers receive a large number of records while other reducers receive very few.
- Reducers handling large partitions take longer to complete, while other reducers remain idle.
- This creates a load imbalance and increases the overall job execution time.
- Skewed keys can cause a particular reducer to become overloaded.
- An inappropriate number of reducers can also contribute to inefficient resource utilization.
- In cloud environments, the imbalance can leave some virtual machines highly utilized while others remain underutilized.
- Therefore, the problem is mainly related to data distribution and workload balancing, rather than simply a lack of computing resources.

Corrective Actions

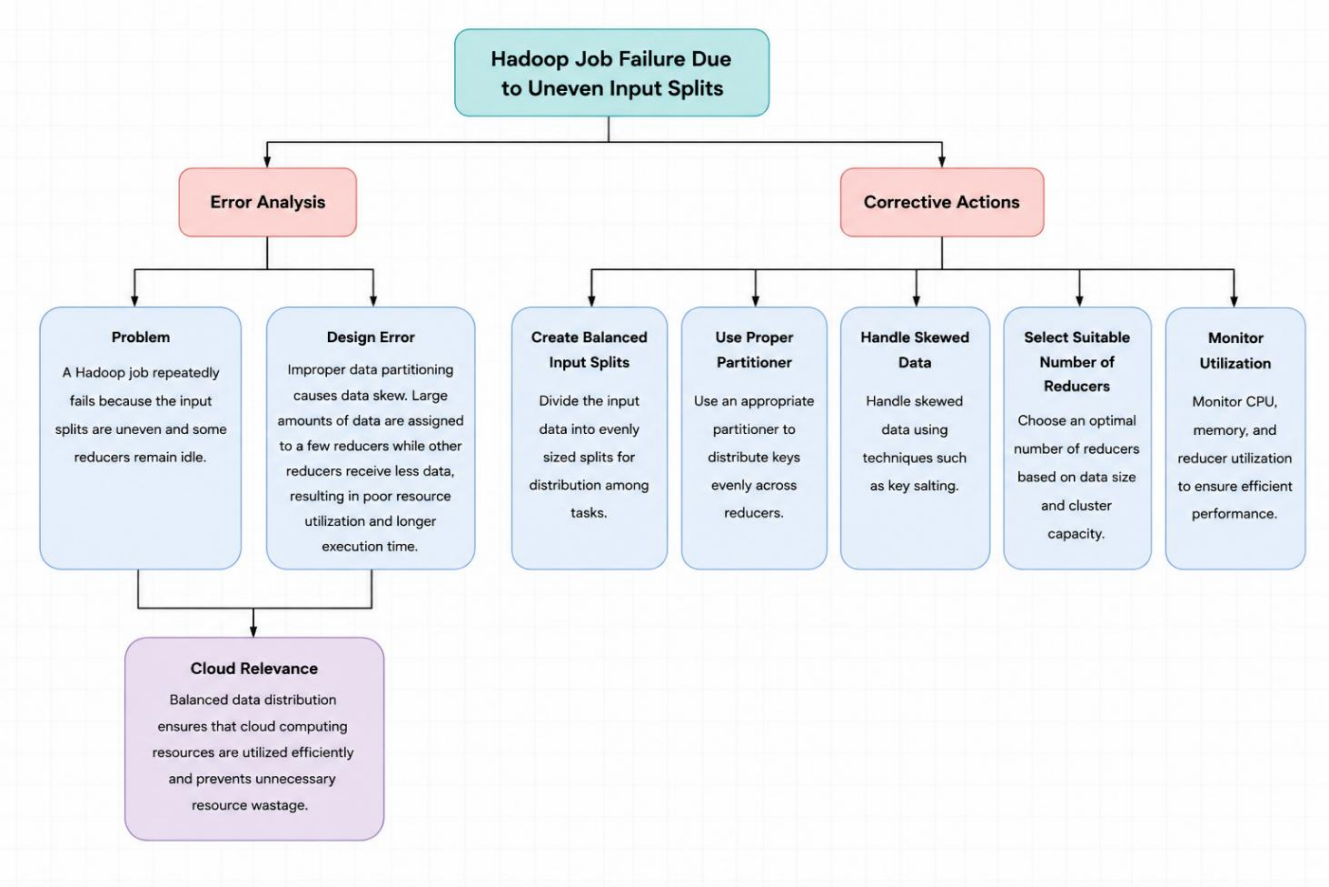
1. Create balanced input splits according to the size and nature of the dataset.
2. Use a suitable partitioner to distribute records evenly among reducers.
3. Identify frequently occurring or skewed keys.
4. Apply key salting to distribute highly skewed keys across multiple reducers.
5. Select an appropriate number of reducers based on the input data size.
6. Repartition the data when significant data skew is detected.
7. Monitor the amount of data processed by each mapper and reducer.
8. Optimize the shuffle and sort phase to reduce unnecessary data transfer.

Cloud Importance

In a cloud environment, Hadoop jobs usually run across multiple virtual machines or cloud nodes. Uneven workload distribution causes some nodes to be overloaded while other nodes remain underutilized.

Proper partitioning helps to:

- Improve parallel processing.
- Increase CPU and memory utilization.
- Reduce job execution time.



Example

For example, if a dataset contains 1 million records and the data is distributed among four reducers, an ideal distribution would give approximately 250,000 records to each reducer. If one reducer receives 700,000 records while the other three receive approximately 100,000 records each, the first reducer will take much longer to finish. The other reducers may remain idle after completing their smaller workloads.

This situation is called data skew.

The Hadoop job failure is mainly caused by uneven input partitioning and data skew. Proper input splits, effective partitioning, key-salting techniques, and suitable reducer configuration can distribute the workload more evenly. This improves Hadoop performance, cloud resource utilization, scalability, and processing efficiency.

2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

An HDFS deployment shows frequent data unavailability after a single DataNode failure. When a node fails, users are unable to access some of the stored data blocks. This indicates that the HDFS deployment does not have sufficient fault tolerance or proper replica management.

Error Analysis

The probable design error is insufficient replication of HDFS data blocks or improper placement of replicas.

- HDFS divides files into blocks and stores copies of these blocks on different DataNodes.
- If the replication factor is too low, a DataNode failure may make the only available copy of a data block inaccessible.
- If multiple replicas are stored on the same DataNode or rack, a single infrastructure failure can affect several copies simultaneously.
- Poor replica placement reduces the fault tolerance of the HDFS cluster.
- Another possible problem is a single NameNode, which can become a single point of failure.
- If the NameNode fails, clients may not be able to access HDFS metadata even when the actual data blocks are available.
- Therefore, the design should provide both data replication and NameNode High Availability.

Corrective Actions

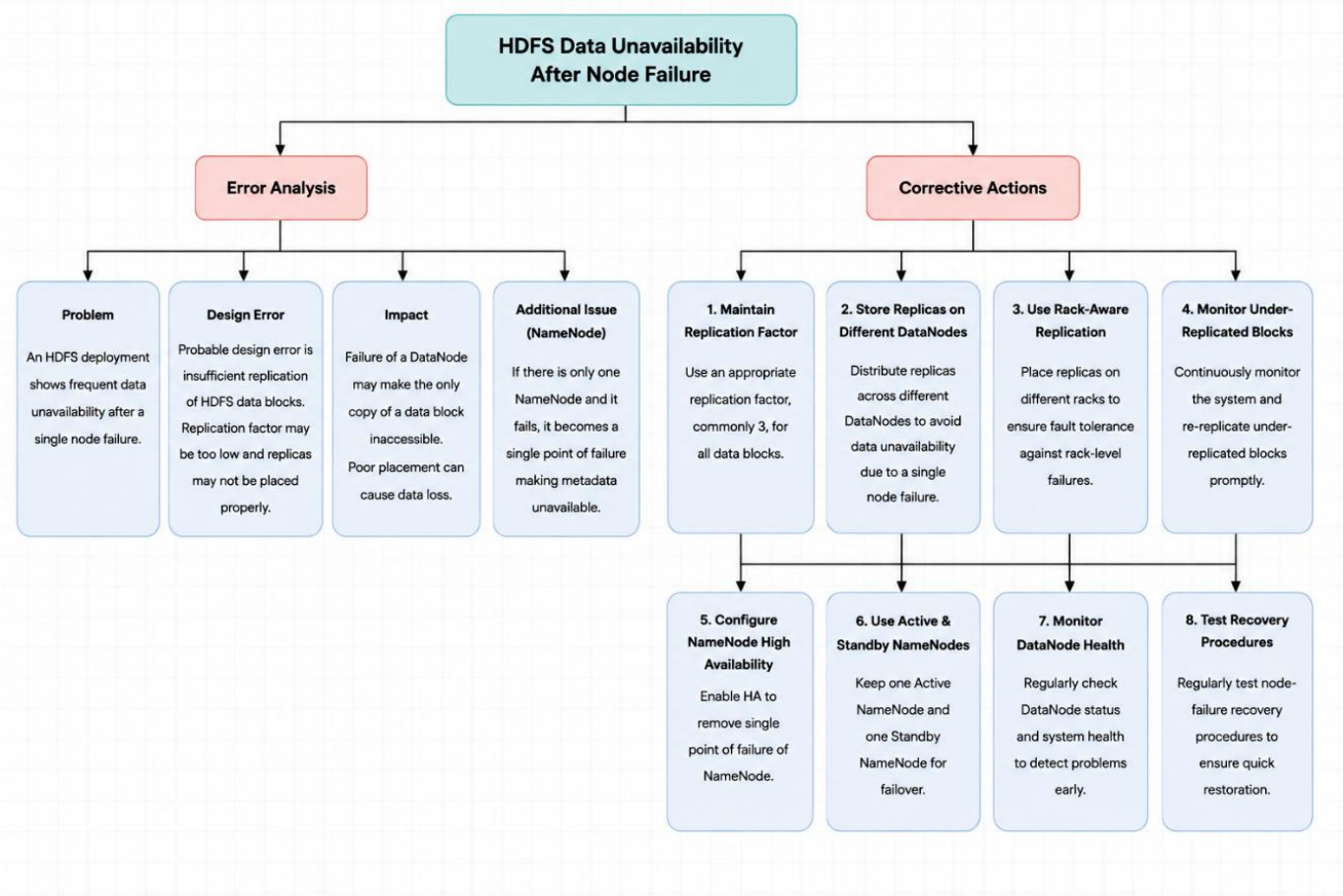
1. Maintain an appropriate replication factor, commonly 3.
2. Store replicas across different DataNodes.
3. Use rack-aware replication to place replicas across different racks.
4. Monitor under-replicated and missing data blocks.
5. Automatically re-replicate blocks when a DataNode fails.
6. Configure NameNode High Availability.
7. Use Active and Standby NameNodes.
8. Monitor DataNode health and cluster status regularly.

Cloud Importance

Cloud infrastructure is distributed across multiple servers, virtual machines, and storage nodes. Hardware or virtual machine failures can occur at any time.

HDFS replication helps to:

- Provide fault tolerance.
- Maintain data availability after node failure.
- Improve data durability.
- Prevent loss of important data.



Example

For example, suppose an HDFS file is divided into data blocks and the replication factor is **3**. Each block is stored on three different DataNodes.

If DataNode 1 fails, copies of the same block are still available on DataNode 2 and DataNode 3. HDFS can serve the data from one of the available replicas and create a new replica to restore the required replication level.

NameNode High Availability

The NameNode maintains important HDFS metadata, such as file names, directory information, and the locations of data blocks.

A single NameNode can become a single point of failure. To avoid this problem, HDFS can use:

Active NameNode → Standby NameNode

3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. The same supplier, customer, transaction, or business record may be inserted multiple times, resulting in incorrect analytics and increased storage usage.

Error Analysis

The main design error is the absence of duplicate detection and idempotent processing in the data ingestion pipeline.

- ETL pipelines extract data from different sources and transfer it to the analytics database.
- If the same source record is processed more than once, duplicate records can be created.
- Workflow retries after temporary failures can cause the same record to be inserted again.
- Network failures may cause a transaction to be repeated.
- Repeated file ingestion can process the same input file multiple times.
- Duplicate records may already exist in the source system.
- Incorrect NiFi flow configuration can cause the same data to pass through the pipeline repeatedly.
- If the destination database does not have a unique constraint, it may accept identical records.
- Without proper checkpoints, the system may not know which records have already been processed.
- Therefore, the ETL pipeline should be designed to safely handle retries and repeated processing.

Corrective Actions

1. Assign a unique identifier to every record.
2. Use NiFi duplicate-detection mechanisms.
3. Check whether a record already exists before inserting it.
4. Apply PRIMARY KEY or UNIQUE constraints in the database.
5. Use upsert operations instead of blindly inserting records.
6. Maintain processing checkpoints and transaction information.
7. Design the ETL pipeline for idempotent processing.
8. Monitor failed and retried transactions.
9. Maintain proper data validation before loading records.

10. Keep logs to identify repeated or failed data processing.

Example

Suppose a customer record with the ID **C101** is received by Apache NiFi.

During the first processing attempt:

C101 → Transform → Database → Record Stored

If a network failure occurs after the database operation and NiFi retries the same record, the pipeline may process:

C101 → Transform → Database → Duplicate Record

If the database has no unique constraint or duplicate check, both records will be stored.

With an idempotent design, the system checks the unique ID before insertion:

C101 → Check Existing Record → Already Exists → Skip/Update

This prevents duplicate data.

Role of Apache NiFi

Apache NiFi provides a visual data-flow environment for collecting, transforming, routing, and transferring data between systems.

A properly designed NiFi workflow should:

- Validate incoming data.
- Generate or identify unique record IDs.
- Detect duplicate records.
- Handle failures and retries safely.
- Route invalid or duplicate records appropriately.
- Maintain processing status and logs.
- Transfer only valid records to the analytics store.

Idempotent Processing

Idempotent processing means that processing the same record multiple times produces the same final result rather than creating multiple copies.

For example:

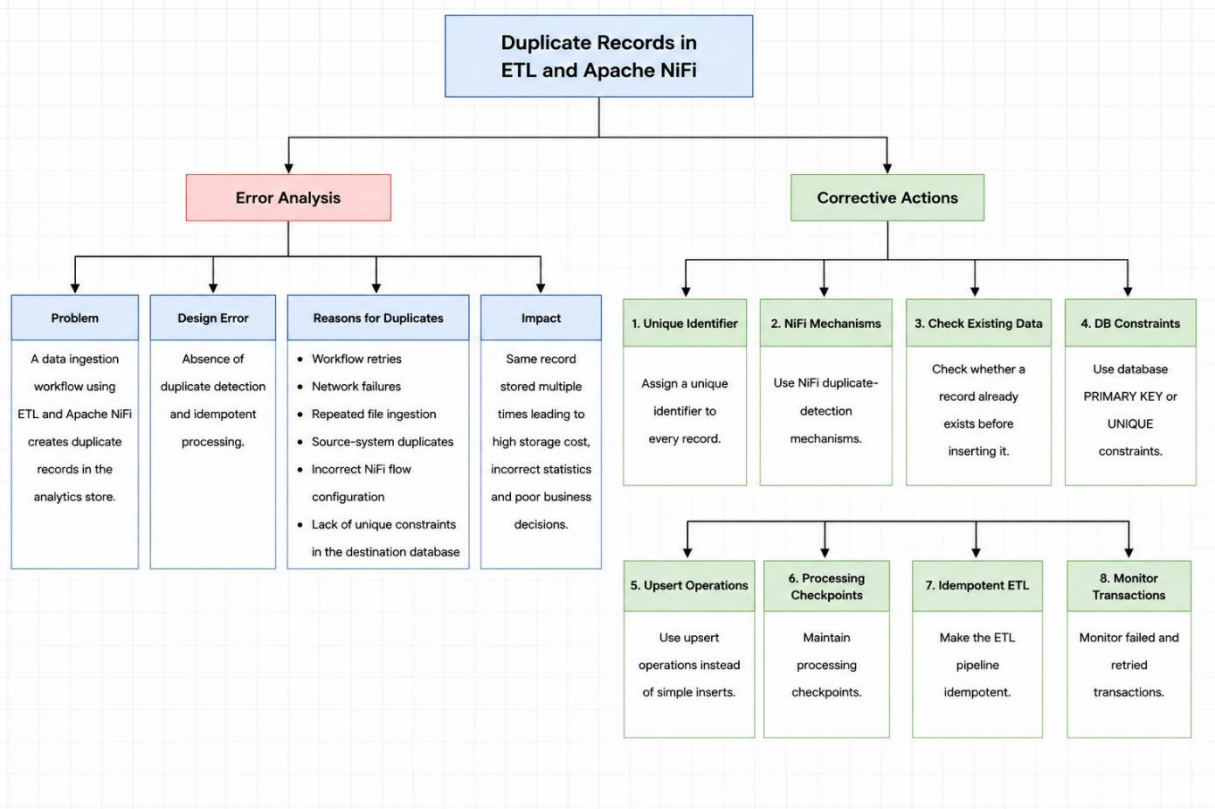
Without Idempotency:

Record → Insert → Insert Again → Duplicate

With Idempotency:

Record → Check ID → Existing? → Update/Ignore

This is especially important in cloud environments where distributed systems may automatically retry failed operations.



The duplicate-record problem is mainly caused by repeated processing, missing duplicate detection, lack of unique constraints, and non-idempotent ETL design. Using unique identifiers, NiFi duplicate detection, database constraints, upsert operations, checkpoints, and proper monitoring can maintain data consistency, reduce cloud storage costs, and improve the reliability of analytics systems.

4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

A YARN cluster suffers from resource starvation when multiple users submit jobs at the same time. Some jobs consume a large amount of CPU and memory, leaving insufficient resources for other users' applications.

Error Analysis

The main design error is improper resource scheduling and allocation among multiple users.

- YARN manages cluster resources such as CPU, memory, and processing capacity.
- If resources are not distributed fairly, one user or application can consume most of the available resources.
- Resource-intensive jobs may occupy the cluster for a long time.
- Other users' jobs remain in the queue and experience long waiting times.
- Lack of queue configuration can allow one user to dominate cluster resources.
- Missing CPU and memory limits can result in excessive resource consumption.
- Poor resource monitoring can make it difficult to identify overloaded applications.
- In a cloud environment, inefficient allocation can cause resource wastage and increased infrastructure costs.
- Therefore, the cluster requires proper scheduling, queue management, quotas, and resource limits.

Corrective Actions

1. Configure YARN Capacity Scheduler or Fair Scheduler.
2. Create separate queues for different users or departments.
3. Set appropriate CPU and memory limits.
4. Configure user and queue resource quotas.
5. Assign priorities to important applications.
6. Prevent a single user from consuming all available resources.
7. Monitor CPU, memory, and queue utilization.
8. Dynamically scale cloud resources when workload increases.
9. Set suitable limits for individual applications.
10. Regularly review resource usage and adjust queue capacities.

Cloud Importance

In cloud environments, multiple users and applications often share the same infrastructure. Proper YARN resource management helps to:

- Provide fair resource allocation.
- Improve CPU and memory utilization.
- Reduce application waiting time.
- Prevent resource starvation.
- Support multiple users efficiently.

Example

Suppose a YARN cluster has 100 GB memory and 20 CPU cores available.

If User A submits several large jobs and consumes 80 GB memory and 16 CPU cores, only a small amount of resources remains for Users B and C.

As a result:

User A → Large Jobs → Most Resources

Users B & C → Insufficient Resources → Jobs Pending

With proper queue management, the resources can instead be distributed among users according to configured limits.

Role of YARN Scheduler

The YARN scheduler is responsible for allocating cluster resources to applications.

A suitable scheduler can divide resources into different queues:

- **Queue 1 → User A**
- **Queue 2 → User B**
- **Queue 3 → User C**

Each queue receives a defined amount of CPU and memory. This prevents a single user from consuming the entire cluster.

Capacity Scheduler

The Capacity Scheduler allocates a guaranteed capacity to different queues. It is useful when organizations need predictable resource allocation among departments or users.

Fair Scheduler

The Fair Scheduler attempts to distribute resources fairly among running applications so that no user or application unnecessarily dominates the cluster.

Dynamic Scaling

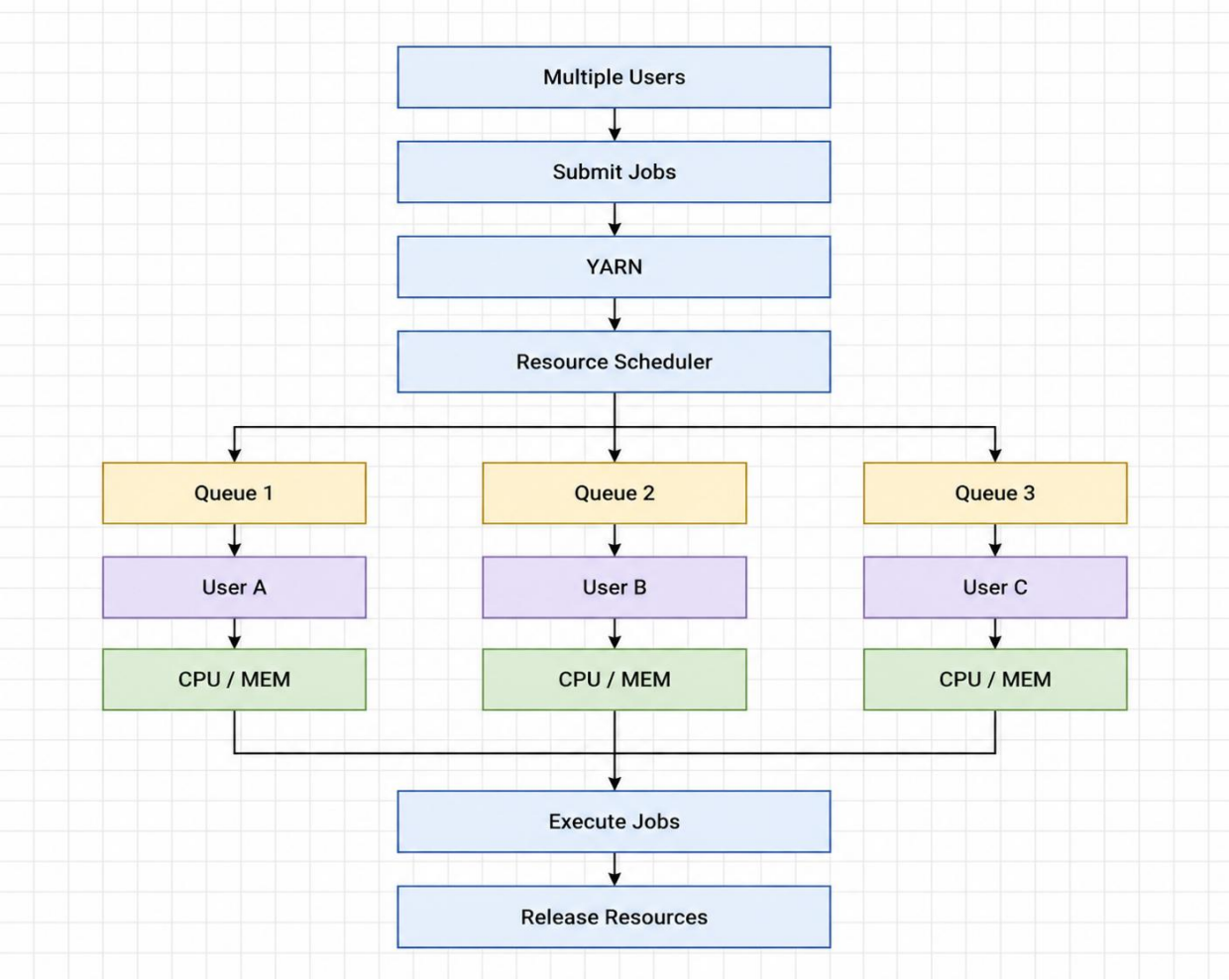
Cloud environments can increase or decrease computing resources according to workload requirements.

For example:

High Workload → Increase Resources → More Jobs Processed

Low Workload → Reduce Resources → Lower Cloud Cost

This helps the system handle sudden increases in job submissions while maintaining efficient resource utilization.



YARN resource starvation is mainly caused by poor scheduling, lack of resource limits, and unfair allocation of CPU and memory. Using Capacity or Fair Scheduling, separate queues, quotas, priorities, monitoring, and dynamic cloud scaling can provide fair resource allocation, better cluster utilization, scalability, reduced waiting time, and lower cloud costs.

5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

A backup strategy for distributed storage restores outdated data after system recovery. The recovered data does not contain the latest changes made before the system failure.

Error Analysis

The probable design flaw is the use of stale backups or unsynchronized replicas.

- Backups may be created too infrequently, so recent changes are not included.
- Replication may be delayed, causing replicas to contain older versions of data.
- The system may restore an old backup instead of the latest valid recovery point.
- Replication alone should not be considered a complete backup strategy.
- Accidental deletion or corruption can also be replicated to other copies.
- Lack of backup verification can result in restoring an incomplete or outdated backup.
- Failure to maintain multiple recovery points makes it difficult to recover the latest valid version.
- Inadequate RPO (Recovery Point Objective) planning can result in significant data loss.
- Poor RTO (Recovery Time Objective) planning can increase system downtime.
- Therefore, the backup design should combine regular backups, versioning, monitoring, and tested recovery procedures.

Corrective Actions

1. Perform regular and scheduled backups.
2. Maintain multiple versions of backups.
3. Use full and incremental backups.
4. Verify the freshness and integrity of backups.
5. Monitor backup completion and replication synchronization.
6. Maintain multiple recovery points.
7. Define a suitable RPO based on business requirements.
8. Define a suitable RTO for faster recovery.
9. Regularly test the data restoration process.
10. Store backup copies separately from the production environment.

Cloud Importance

Cloud applications depend on reliable storage and continuous availability. A proper backup and disaster recovery strategy helps to:

- Protect data from hardware failures.
- Recover from data corruption.
- Prevent permanent data loss.
- Recover from accidental deletion.
- Reduce service downtime.
- Maintain business continuity.
- Improve data durability.
- Provide reliable disaster recovery.

Example

Suppose a company's database is backed up every 24 hours.

If the last backup was taken at 12:00 AM and the system fails at 10:00 PM, the backup may not contain the transactions performed during the day.

Therefore:

Latest Data → 10:00 PM

Available Backup → 12:00 AM

Recovery → Outdated Data

A better strategy is to use frequent incremental backups or snapshots so that the latest valid recovery point is available.

RPO and RTO

RPO – Recovery Point Objective

RPO defines the **maximum acceptable amount of data that can be lost** after a failure.

For example, an RPO of 1 hour means the organization should be able to recover data with no more than approximately one hour of data loss.

RTO – Recovery Time Objective

RTO defines the **maximum acceptable time required to restore the system** after a failure.

For example, an RTO of 2 hours means the system should be restored and operational within approximately two hours.

Backup and Recovery Strategy

A reliable cloud backup system should follow:

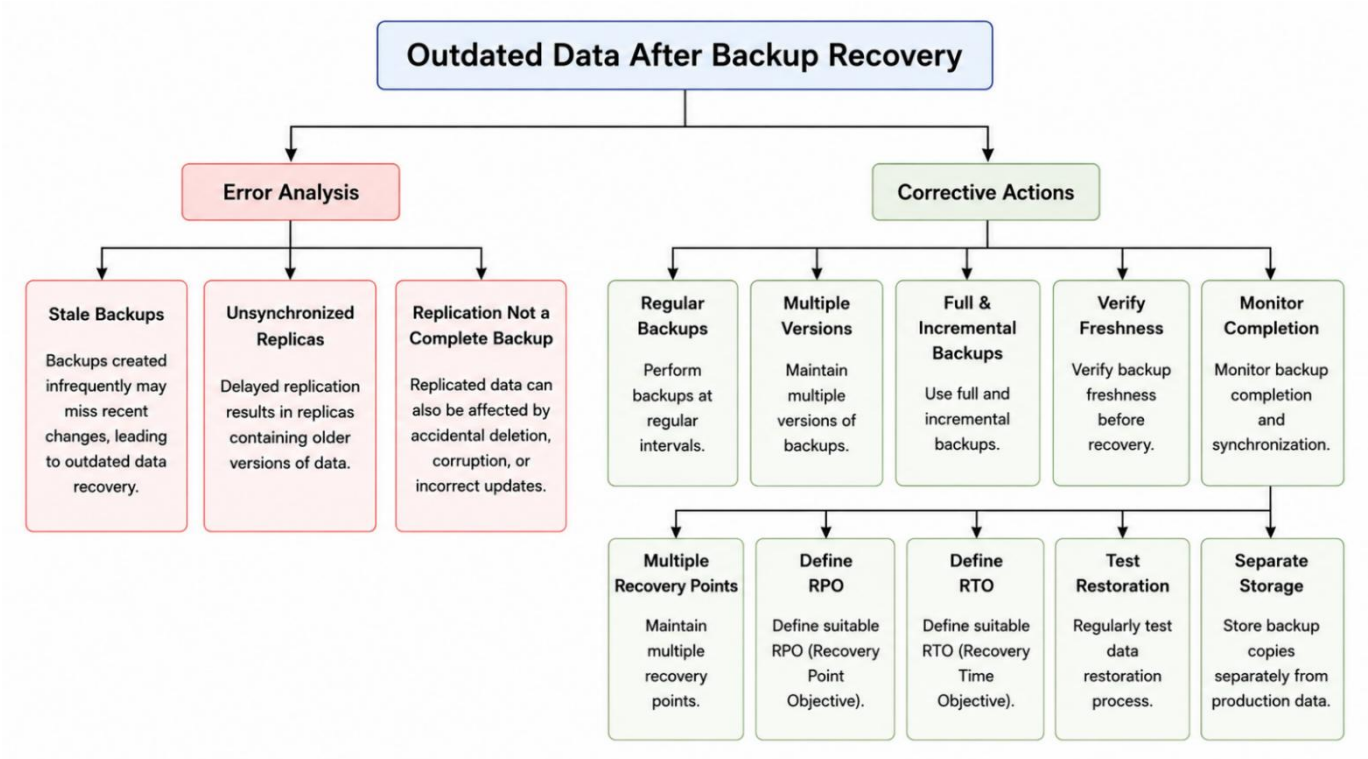
Production Data → Regular Backup → Versioned Storage → Failure → Latest Valid Backup → Restore → Verify Data → System Recovery

This ensures that recovery uses the latest reliable copy rather than an outdated backup.

Replication vs Backup

Replication and backup serve different purposes.

Replication mainly provides availability by maintaining copies of data across different systems.



The outdated-data problem is mainly caused by stale backups, delayed replication, insufficient backup frequency, and poor recovery planning. Using regular versioned backups, independent storage, backup verification, suitable RPO/RTO values, and regular restoration testing can provide reliable disaster recovery, data durability, business continuity, and protection against data loss.